

In practice, it's easier to calculate the cosine of the angle between two vectors than it is to calculate the angle itself. This is not a problem, since we can conclude that if the cosine of the angle between two vectors is large, then the angle itself is small and vice versa. In Python the `scipy` module contains a submodule called `spatial`, which contains a function for calculating the cosines of angles between vectors. We can use the functionality in `spatial` to calculate cosines between each chapter description vector and query vector, by using a list comprehension:

```
row_similarities = [1 - spatial.distance.cosine(tfidf_reports[x],tfidf_question)
for x in \range(len(tfidf_reports)) ]
```

When we print out the `row_similarities` variable, we see the following vector:

```
[0.0, 0.0, 0.0, 0.3393118510377361, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

In this case, only the fourth element is greater than zero, meaning that only the fourth chapter description vector has any angular proximity to our query vector. In general, we can automatically find which row has the highest cosine similarity:

```
print(alldocuments[np.argmax(row_similarities)])
```

This gives us the chapter the chatbot thinks we're looking for:

```
Chapter 4. Sorting and searching, including merge sort, and algorithm runtime.
```

[Listing 11-1](#) puts the chatbot's simple functionality into a function.

```
def chatbot(query,allreports):
    clf = TfidfVectorizer(ngram_range = (1, 1),tokenizer = normalize, stop_words
= 'english')
    clf.fit(allreports)
    tfidf_reports = clf.transform(allreports).todense()
    tfidf_question = clf.transform([query]).todense()
    row_similarities = [1 -
spatial.distance.cosine(tfidf_reports[x],tfidf_question) for x in
\range(len(tfidf_reports)) ]
    return(allreports[np.argmax(row_similarities)])
```
